The AI Operating System, Part III: Memory Layer — Typed Filesystem for Persistent Agent Cognition

Matthew Long
The YonedaAI Collaboration
YonedaAI Research Collective
Chicago, IL
matthew@yonedaai.com · https://yonedaai.com

March 10, 2026

Abstract

Autonomous AI agents require persistent memory that transcends session boundaries, yet no analogue of the operating-system filesystem has been formalized for agent cognition. Traditional filesystems store untyped byte streams; agent memory demands typed schemas, causal versioning, semantic search, and graph-structured lineage. We introduce a *categorical memory model* in which memory is a functor $M : \mathcal{S} \to \mathcal{St}$ from a schema category $\mathcal{S}$ (whose objects are typed schemas and morphisms are schema transformations) to a storage category $\mathcal{St}$ (whose objects are storage backends and morphisms are queries). Type safety is expressed as natural transformations between memory functors; versioning is a temporal functor $V : \mathsf{Timeline} \to \mathcal{St}$; and graph lineage is a 2-categorical structure layered over the base categories. We prove that the generic typed wrapper $\mathsf{Mem}[S]$ is a representable functor (a Yoneda-lemma connection), that type structure is preserved under storage round-trips, and that the memory operations `evolve`, `merge`, and `split` form a monad on the memory category. The reference implementation, ContextFS, realizes this framework with 24 memory-type variants, a multi-backend storage router (SQLite, ChromaDB, FalkorDB, PostgreSQL), vector clocks for causal consistency, content hashing for deduplication, and an MCP server for universal client access. We provide a production-quality Elixir/OTP implementation using ETS for working memory and Mnesia for persistent knowledge, demonstrating that the categorical abstractions yield both formal guarantees and practical performance. Case studies in agent learning, cross-session continuity, and multi-agent shared knowledge validate the framework against real workloads.

Keywords: agent memory, typed filesystem, category theory, knowledge representation, persistent cognition, OTP, MCP, semantic search

Contents

1 Introduction

The first two installments of this series established the kernel architecture and the scheduler for an AI Operating System. Yet an operating system without a memory subsystem is merely a stateless dispatcher: it can schedule computations but cannot accumulate knowledge across them. Every traditional OS dedicates a substantial fraction of its complexity to memory management—virtual memory, page tables, filesystem hierarchies, caching layers—because persistent, organized storage is the foundation upon which all higher-level abstractions rest.

For autonomous AI agents, the memory problem is simultaneously more constrained and more ambitious than its classical counterpart. More constrained because agents do not need byte-addressable random access to flat address spaces; more ambitious because agent memory must support:

(i) **Typed schemas**: each memory entry conforms to a domain-specific type, not a raw byte stream.

(ii) **Causal versioning**: memory evolves over time, and the system must track *why* each change occurred (observation, inference, correction, decay).

(iii) **Semantic search**: retrieval by meaning, not merely by address or filename.

(iv) **Graph lineage**: memories relate to one another through typed edges—one memory may supersede, contradict, reference, or evolve into another.

(v) **Multi-backend coordination**: different storage engines excel at different query patterns (relational, vector, graph), and the memory layer must route transparently.

Traditional filesystems fail on every count. POSIX provides `open`/`read`/`write`/`close` on untyped byte sequences organized in a tree hierarchy. Plan 9's union mounts and synthetic filesystems (**?**) introduced compositional namespace ideas but remained fundamentally untyped. Semantic filesystems (**?**) added metadata-based retrieval but lacked formal type guarantees. None of these systems address versioning with causal annotations, graph-structured relationships, or the combination of exact and approximate (embedding-based) retrieval that agent cognition requires.

This paper formalizes agent memory using category theory and provides a reference implementation called ContextFS. The categorical framework yields three concrete benefits:

1. **Compositionality**: memory operations compose cleanly because they are morphisms in well-defined categories.

2. **Type safety**: the functor $M\colon \mathcal{S} \to \mathcal{S}t$ preserves type structure, and we prove round-trip fidelity.

3. **Extensibility**: new schema types, storage backends, and relationship kinds can be added as new objects and morphisms without modifying the core framework.

The structure of the paper is as follows. Section **??** surveys relevant background in OS memory management, knowledge representation, and category theory. Section **??**

develops the formal memory model. Section **??** details the type system built on schema-indexed memory. Section **??** formalizes versioning and lineage. Section **??** presents the multi-backend storage architecture. Section **??** establishes reliability guarantees. Section **??** provides the Elixir/OTP implementation. Section **??** validates the framework through case studies. Section **??** compares to related work, and Section **??** concludes.

2 Background

2.1 Virtual Memory and Filesystems

The virtual memory subsystem of a modern OS provides each process with an isolated address space mapped onto physical memory and secondary storage through page tables (**?**). The key abstraction is *indirection*: processes reference virtual addresses, and the memory management unit (MMU) translates these to physical locations. This indirection enables demand paging, copy-on-write, memory-mapped files, and shared memory regions.

Filesystems layer a persistent namespace on top of block devices. The Unix filesystem (**?**) introduced the inode/directory model: files are sequences of bytes identified by inodes, organized in a tree of directories. The Virtual Filesystem Switch (VFS) in Linux (**?**) provides a uniform interface over heterogeneous filesystem implementations—ext4, XFS, NFS, FUSE—through a common set of operations (`inode_operations`, `file_operations`, `super_operations`).

Plan 9 (**?**) radicalized the "everything is a file" philosophy: network connections, process state, graphics, and device control are all accessed through filesystem operations on synthetic file servers. The `/proc` filesystem, `/net`, and `/dev` present dynamic system state as navigable file trees. Union mounts allow multiple file trees to be overlaid into a single namespace.

These designs are profoundly influential, yet they share a fundamental limitation: *files are untyped byte streams*. The filesystem knows nothing about the structure of what it stores. All interpretation is deferred to applications.

2.2 Knowledge Representation and Semantic Memory

The knowledge representation (KR) community has long studied how to organize structured information for intelligent systems. Frame systems (**?**) organize knowledge around stereotypical situations with slots and defaults. Semantic networks (**?**) represent concepts as nodes and relationships as labeled edges. Description logics (**?**) provide formal semantics with decidable reasoning for ontological knowledge.

More recently, knowledge graphs (**?**) have become the dominant paradigm for structured knowledge storage, exemplified by Wikidata, Google's Knowledge Graph, and domain-specific graphs in biomedicine and finance. Knowledge graphs represent entities as nodes and relationships as typed edges, with schemas defined by ontologies.

The connection to agent memory is direct: an agent's accumulated knowledge forms a personal knowledge graph, with the crucial addition that this graph *evolves over time* as the agent observes, infers, corrects, and forgets.

2.3 Embedding Spaces and Semantic Search

Transformer-based language models produce dense vector representations (embeddings) that capture semantic similarity (**??**). Retrieval-augmented generation (RAG) (**?**) exploits this by storing text chunks as vectors and retrieving relevant context at query time through approximate nearest neighbor (ANN) search (**?**).

For agent memory, embeddings provide a crucial capability that neither filesystems nor knowledge graphs offer natively: *retrieval by meaning.* An agent can recall memories semantically related to its current context without knowing exact keys or traversal paths. This is analogous to human associative memory (**?**), where recall is triggered by semantic similarity rather than explicit addressing.

2.4 Category Theory Preliminaries

We assume familiarity with categories, functors, and natural transformations at the level of Mac Lane (**?**). A *category* $\mathcal{C}$ consists of objects $\mathrm{ob}(\mathcal{C})$, morphisms $\mathrm{mor}(\mathcal{C})$ with source and target maps, an associative composition, and identity morphisms. A *functor* $F\colon \mathcal{C} \to \mathcal{D}$ maps objects to objects and morphisms to morphisms, preserving composition and identities. A *natural transformation* $\eta\colon F \Rightarrow G$ between parallel functors $F, G\colon \mathcal{C} \to \mathcal{D}$ is a family of morphisms $\eta_c\colon F(c) \to G(c)$ satisfying the naturality square for every morphism in $\mathcal{C}$.

The *Yoneda lemma* states that for any functor $F\colon \mathcal{C}^{\mathrm{op}} \to \mathbf{Set}$ and object $c \in \mathcal{C}$:

$$\mathrm{Nat}(\mathrm{Hom}_{\mathcal{C}}(-, c), F) \cong F(c)$$

A functor is *representable* if it is naturally isomorphic to $\mathrm{Hom}_{\mathcal{C}}(-, c)$ for some object c. A *monad* on $\mathcal{C}$ is an endofunctor $T\colon \mathcal{C} \to \mathcal{C}$ equipped with natural transformations $\eta\colon \mathrm{id}_{\mathcal{C}} \Rightarrow T$ (unit) and $\mu\colon T^2 \Rightarrow T$ (multiplication) satisfying associativity and unit laws.

A *2-category* extends a category by adding 2-morphisms (morphisms between morphisms), with two composition operations (horizontal and vertical) satisfying interchange laws (**?**). We will use 2-categorical structure to formalize relationships between memory lineage paths.

3 Memory Model

We now develop the formal categorical framework for agent memory. The central construction is a functor from a category of typed schemas to a category of storage backends, with additional structure for versioning and lineage.

3.1 The Schema Category

Definition 3.1 (Schema Category $\mathcal{S}$)**.** The *schema category* $\mathcal{S}$ has:

- **Objects**: typed schemas $S_1, S_2, \ldots$, each specifying a set of typed fields with validation constraints. In ContextFS, these include `BaseSchema` and its 24 domain-specific descendants (see Section **??**).
- **Morphisms**: schema transformations $f\colon S_i \to S_j$, which are structure-preserving maps between schemas. A morphism f maps each field of S_i to a (possibly computed) field of S_j, preserving type compatibility.

- **Identity**: the identity transformation $\mathrm{id}_S \colon S \to S$.
- **Composition**: given $f \colon S_1 \to S_2$ and $g \colon S_2 \to S_3$, the composite $g \circ f \colon S_1 \to S_3$ applies f then g.

The schema category has a rich subcategory structure. Let $\mathcal{S}_{\text{core}}$ denote the full subcategory on the four core memory types (FACT, DECISION, PROCEDURAL, EPISODIC); $\mathcal{S}_{\text{ext}}$ the extended types (TODO, ISSUE, API, SCHEMA); and $\mathcal{S}_{\text{wf}}$ the workflow types (WORKFLOW, TASK, STEP, AGENT_RUN). These subcategories compose:

$$\mathcal{S} = \mathcal{S}_{\text{core}} \cup \mathcal{S}_{\text{ext}} \cup \mathcal{S}_{\text{wf}} \cup \mathcal{S}_{\text{meta}} \cup \mathcal{S}_{\text{sys}}$$

where $\mathcal{S}_{\text{meta}}$ and $\mathcal{S}_{\text{sys}}$ contain metadata and system-level schemas respectively.

Remark 3.2. The `BaseSchema` in ContextFS serves as an initial object in $\mathcal{S}$ in the sense that every domain schema admits a unique embedding from `BaseSchema` (via inheritance). The `_schema_name` class variable acts as a tagging morphism that distinguishes objects.

3.2 The Storage Category

Definition 3.3 (Storage Category $\mathcal{St}$)**.** The *storage category* $\mathcal{St}$ has:

- **Objects**: storage backends $B_1, B_2, \ldots$, each providing a set of capabilities (save, recall, search, delete, update). In ContextFS, these include SQLite (local relational), ChromaDB (vector embeddings), FalkorDB (graph), and PostgreSQL (cloud relational).
- **Morphisms**: queries $q \colon B_i \to B_j$, representing data-flow operations that may retrieve from one backend and store in another, or translate between storage representations.
- **Identity**: the no-op query id_B that returns data unchanged.
- **Composition**: pipeline composition of queries.

Each backend B comes equipped with a *capability set* $\mathrm{Cap}(B) \subseteq \{\texttt{save}, \texttt{recall}, \texttt{search}, \texttt{delete}, \texttt{upda}$
This defines a functor $\mathrm{Cap} \colon \mathcal{St} \to \mathcal{P}(\mathrm{Ops})$ to the powerset category of operations.

3.3 The Memory Functor

Definition 3.4 (Memory Functor)**.** The *memory functor* $M \colon \mathcal{S} \to \mathcal{St}$ maps:

- Each schema $S \in \mathrm{ob}(\mathcal{S})$ to a storage representation $M(S) \in \mathrm{ob}(\mathcal{St})$, which is the backend (or product of backends) where instances of S are persisted.
- Each schema transformation $f \colon S_1 \to S_2$ to a storage operation $M(f) \colon M(S_1) \to M(S_2)$ that transforms stored instances accordingly.

subject to the functor laws: $M(\mathrm{id}_S) = \mathrm{id}_{M(S)}$ and $M(g \circ f) = M(g) \circ M(f)$.

The generic typed wrapper $\mathsf{Mem}[S]$ in ContextFS is precisely the action of this functor on a specific schema object S: it produces a typed container that mediates between the schema type and the storage representation.

The following commutative diagram captures the fundamental structure:

$$\begin{tikzcd} S_1 \arrow[r, "f"] \arrow[d, "M"'] & S_2 \arrow[d, "M"] \\ M(S_1) \arrow[r, "M(f)"'] & M(S_2) \end{tikzcd}$$

3.4 Representability and the Yoneda Connection

We now establish the key theoretical result connecting $\mathsf{Mem}[S]$ to the Yoneda lemma.

Consider the contravariant functor $\mathsf{Mem}[-]\colon \mathcal{S}^{\mathrm{op}} \to \mathbf{Set}$ that sends each schema S to the set of all memory instances of type S (i.e., the set $\mathsf{Mem}[S]$.instances), and each schema morphism $f\colon S_1 \to S_2$ to the pullback function $f^*\colon \mathsf{Mem}[S_2]$.instances $\to$ $\mathsf{Mem}[S_1]$.instances.

Theorem 3.5 ($\mathsf{Mem}[S]$. *isRepresentable]Thefunctor Mem[-]*: $\mathcal{S}^{\mathrm{op}} \to \mathbf{Set}$ *is representable. Specifically, there exists a universal schema* $U \in \mathrm{ob}(\mathcal{S})$ *such that:*

$$\mathsf{Mem}[-] \cong \mathrm{Hom}_{\mathcal{S}}(-, U)$$

The representing object U *is the* `BaseSchema` *equipped with all optional extensions.*

Proof. By the Yoneda lemma, $\mathrm{Nat}(\mathrm{Hom}_{\mathcal{S}}(-, U), \mathsf{Mem}[-]) \cong \mathsf{Mem}[U]$. We must show that there exists a universal element $u \in \mathsf{Mem}[U]$ such that the induced natural transformation is an isomorphism.

Let $U =$ `BaseSchema` with the universal schema carrying all field types as optional. For any schema S and instance $m \in \mathsf{Mem}[S]$, the unique morphism $\phi_m\colon S \to U$ embeds the instance into the universal schema (setting unused fields to their zero values). This defines a natural bijection:

$$\mathsf{Mem}[S] \ni m \longmapsto (\phi_m, m|_{\mathrm{base}}) \in \mathrm{Hom}_{\mathcal{S}}(S, U) \times \mathsf{Mem}[U]_{\mathrm{base}}$$

Since every schema transformation $f\colon S_1 \to S_2$ factors through U via the respective embeddings, and the functor $\mathsf{Mem}[-]$ respects this factorization (by the type-preservation property of the storage layer), the natural transformation is an isomorphism. The Yoneda embedding ensures faithfulness. □

Corollary 3.6. *The set of all memory instances of schema type* S *is fully determined by the morphisms from* S *into the universal schema* U*:*

$$|\mathsf{Mem}[S]| = |\mathrm{Hom}_{\mathcal{S}}(S, U)|$$

3.5 Versioning as Temporal Functor

Definition 3.7 (Timeline Category $\mathsf{Timeline}$)**.** The *timeline category* $\mathsf{Timeline}$ has:

- **Objects**: timestamps $t_0, t_1, t_2, \ldots$ (elements of a totally ordered set).

- **Morphisms**: a unique morphism $t_i \to t_j$ whenever $t_i \leq t_j$ (making $\mathsf{Timeline}$ a thin category, i.e., a preorder viewed as a category).

Definition 3.8 (Versioning Functor)**.** The *versioning functor* $V\colon \mathsf{Timeline} \to \mathcal{S}t$ maps each timestamp t to the state of storage at time t, and each morphism $t_i \leq t_j$ to the sequence of storage operations applied between t_i and t_j.

The versioned memory wrapper $\mathsf{VMem}[S]$ in ContextFS is the composite functor:

$$\mathsf{VMem}[S] = V \circ M|_S \colon \mathsf{Timeline} \to \mathcal{S}t$$

where $M|_S$ restricts the memory functor to the single schema S. Each version carries a `ChangeReason` annotation drawn from the reason category (Definition **??** in Section **??**).

4 Type System

4.1 Schema-Indexed Types

The fundamental type-theoretic construction in ContextFS is the *schema-indexed memory wrapper* $\mathsf{Mem}[S]$, parameterized by a schema type S. This is a dependent type in the sense that the operations available on a memory instance depend on the schema it wraps.

Definition 4.1 (Schema-Indexed Memory)**.** For a schema $S \in \mathrm{ob}(\mathcal{S})$, the type $\mathsf{Mem}[S]$ provides:

- $\mathsf{Mem}[S].\texttt{create}(d\colon S) \to \mathsf{Mem}[S]$: construct a new memory wrapping validated data of type S.
- $\mathsf{Mem}[S].\texttt{wrap}(d\colon S) \to \mathsf{Mem}[S]$: wrap existing data, assigning a fresh identifier and timestamp.
- $\mathsf{Mem}[S].\texttt{data} \to S$: extract the typed payload.
- $\mathsf{Mem}[S].\texttt{id} \to \mathrm{UUID}$: unique identifier.
- $\mathsf{Mem}[S].\texttt{schema_name} \to \mathrm{String}$: the schema's type tag (from S's `_schema_name` class variable).

The key property is that $\mathsf{Mem}[S]$ enforces type safety at two levels:

1. **Compile-time** (via mypy in the Python reference, or dialyzer/typespecs in Elixir): the generic parameter S is propagated through all operations, preventing type confusion.
2. **Runtime** (via Pydantic validators in Python, or struct enforcement in Elixir): data is validated against the schema on every create, wrap, and update operation.

4.2 The 24 Memory Types

ContextFS defines 24 memory-type variants organized into five subcategories. Each variant is an object in the schema category $\mathcal{S}$:

Table 1: Memory type taxonomy in ContextFS.

Category	Type	Description
4*Core	`FACT`	Declarative knowledge: verified assertions
	`DECISION`	Decision records with rationale and alternatives
	`PROCEDURAL`	How-to knowledge: steps, preconditions, postconditions
	`EPISODIC`	Experiential records: what happened, when, in what context
4*Extended	`TODO`	Action items with priority and status
	`ISSUE`	Problem records with severity and resolution
	`API`	External API schemas and access patterns
	`SCHEMA`	Schema definitions (meta-level: schemas about schemas)
4*Workflow	`WORKFLOW`	End-to-end process definitions
	`TASK`	Individual units of work within workflows
	`STEP`	Atomic operations within tasks
	`AGENT_RUN`	Execution traces of agent invocations
4*Metadata	`TAG`	Classification labels
	`ANNOTATION`	Human or agent commentary on other memories
	`EMBEDDING`	Vector representations
	`INDEX`	Search index entries
4*System	`CONFIG`	System configuration parameters
	`SESSION`	Agent session state
	`CONTEXT`	Contextual information bundles
	`LOG`	System event records
4*Relational	`PERSON`	People known to the agent
	`PROJECT`	Project-level organizational units
	`ARTIFACT`	Produced outputs (documents, code, analyses)
	`EVENT`	Calendar or timeline events

4.3 Type-Safe Operations as Natural Transformations

The storage operations (save, recall, search, update, delete) are natural transformations between functors defined over the schema category.

Definition 4.2 (Storage Natural Transformation)**.** Let $\mathrm{InMem}, \mathrm{OnDisk} \colon \mathcal{S} \to \mathbf{Set}$ be functors mapping each schema S to, respectively, the set of in-memory instances and the set of on-disk representations. The `save` operation is a natural transformation:

$$\texttt{save} \colon \mathrm{InMem} \Rightarrow \mathrm{OnDisk}$$

satisfying the naturality condition: for every schema morphism $f \colon S_1 \to S_2$,

$$\begin{tikzcd}
\mathrm{InMem}(S_1) \arrow[r, "\texttt{save}_{S_1}"] \arrow[d, "\mathrm{InMem}(f)"'] & \mathrm{OnDisk}(S_1) \arrow[d, "\mathrm{OnDisk}(f)"] \\
\mathrm{InMem}(S_2) \arrow[r, "\texttt{save}_{S_2}"'] & \mathrm{OnDisk}(S_2)
\end{tikzcd}$$

Similarly, `recall` is a natural transformation OnDisk $\Rightarrow$ InMem, and the composition $\texttt{recall} \circ \texttt{save}$ must be naturally isomorphic to the identity—this is the round-trip theorem.

Theorem 4.3 (Type Preservation Under Round-Trip)**.** *For any schema $S \in \mathrm{ob}(\mathcal{S})$ and memory instance $m \in \mathsf{Mem}[S]$:*

$$\texttt{recall}_S(\texttt{save}_S(m)) \cong m$$

More precisely, the composite natural transformation $\texttt{recall} \circ \texttt{save} \colon \mathrm{InMem} \Rightarrow \mathrm{InMem}$ *is naturally isomorphic to the identity natural transformation* $\mathrm{id}_{\mathrm{InMem}}$.

Proof. We must show that for each schema S, the component $(\texttt{recall} \circ \texttt{save})_S = \texttt{recall}_S \circ \texttt{save}_S$ is a bijection on $\mathrm{InMem}(S)$ that commutes with schema transformations.
Bijection: The `save` operation serializes m into a storage representation using schema-specific serializers derived from S's field definitions. The content hash $h(m)$ is computed and stored alongside. The `recall` operation deserializes using the same schema S (resolved via `SchemaRegistry` from the stored `_schema_name` tag). Since serialization and deserialization are inverse operations for the Pydantic/struct validators (which enforce canonical forms), the round-trip is the identity up to isomorphism (UUID and timestamp may be refreshed, but the typed content is preserved).
Naturality: For a schema morphism $f \colon S_1 \to S_2$ and $m \in \mathrm{InMem}(S_1)$:

$$\begin{aligned}
\texttt{recall}_{S_2}(\mathrm{OnDisk}(f)(\texttt{save}_{S_1}(m))) &= \texttt{recall}_{S_2}(\texttt{save}_{S_2}(\mathrm{InMem}(f)(m))) \\
&= \mathrm{InMem}(f)(m)
\end{aligned}$$

where the first equality uses naturality of `save`, and the second uses the bijection property for S_2. □

4.4 The Schema Registry

The `SchemaRegistry` in ContextFS provides runtime type resolution: given a schema name string, it returns the corresponding schema class. Categorically, the registry is a section of the forgetful functor $U \colon \mathcal{S} \to \mathbf{Set}$ that sends each schema to its name:

$$\texttt{SchemaRegistry} \colon \mathrm{Names} \hookrightarrow \mathcal{S}, \qquad U \circ \texttt{SchemaRegistry} = \mathrm{id}_{\mathrm{Names}}$$

This section is injective by construction (schema names are unique), ensuring unambiguous type resolution at runtime.

5 Versioning and Lineage

5.1 The Reason Category

Definition 5.1 (Reason Category Reason)**.** The *reason category* Reason has:

- **Objects**: change reasons {`Observation`, `Inference`, `Correction`, `Decay`}.
- **Morphisms**: subsumption relations between reasons. `Correction` subsumes `Observation` (a correction implies a prior observation was wrong); `Inference` extends `Observation` (an inference is derived from observations); `Decay` is terminal (all knowledge eventually decays).

The reason annotations in $\mathsf{VMem}[S]$ are drawn from $\mathrm{ob}(\mathsf{Reason})$:

```elixir
defmodule MemoryLayer.ChangeReason do
  @type t :: :observation | :inference | :correction | :decay

  def subsumes?(:correction, :observation), do: true
  def subsumes?(:inference, :observation), do: true
  def subsumes?(:decay, _), do: true
  def subsumes?(x, x), do: true
  def subsumes?(_, _), do: false
end
```

Listing 1: ChangeReason enum (Elixir representation).

5.2 Versioned Memory

Definition 5.2 (Versioned Memory)**.** For a schema S, the *versioned memory* $\mathsf{VMem}[S]$ is a functor:

$$\mathsf{VMem}[S] \colon \mathsf{Timeline} \to \mathbf{Set}$$

mapping each timestamp t to the version of the memory at time t, consisting of:

- The typed data $d_t \in S$
- The change reason $r_t \in \mathrm{ob}(\mathsf{Reason})$
- The version number $v_t \in \mathbb{N}$
- The parent version reference (if any)

The version chain forms a path in the timeline category:

$$m_{t_0} \xrightarrow{r_1} m_{t_1} \xrightarrow{r_2} m_{t_2} \xrightarrow{r_3} \cdots$$

where each arrow is labeled with its change reason.

5.3 Graph Relationships as 2-Morphisms

Memory lineage in ContextFS is captured by the `EdgeRelation` enum, which defines typed edges between memories:

Definition 5.3 (Edge Relation Category)**.** The *edge relation category* Edge has:

- **Objects**: memories (instances of $\mathsf{Mem}[S]$ for various S).
- **Morphisms**: typed edges between memories, drawn from the set {`EVOLVED_INTO`, `REFERENCES`, `SUPERSEDES`, `CONTRADICTS`, `SUPPORTS`, `DERIVED_FROM`, `PART_OF`, `TRIGGERS`, `BLOCKED_BY`}.

The 2-categorical structure emerges when we consider paths of edges as 1-morphisms and equivalences between paths as 2-morphisms:

$$\begin{tikzcd} m_1 \arrow[r, bend left, "\texttt{EVOLVED_INTO}"] \arrow[r, bend right, "\texttt{SUPERSEDES}"'] \arrow[r, phantom, "\Downarrow\alpha"] & m_2 \end{tikzcd}$$

Here α is a 2-morphism asserting that "evolved into" and "supersedes" are equivalent lineage paths between m_1 and m_2 in a particular context. This 2-categorical viewpoint allows the system to reason about the *equivalence of different lineage narratives.*

5.4 Memory Operations as a Monad

Proposition 5.4 (Memory Operations Monad)**.** *The memory operations* `evolve`*,* `merge`*, and* `split` *define a monad* (T, η, μ) *on the category of memory instances* $\mathcal{M}$*:*

- *The endofunctor* $T\colon \mathcal{M} \to \mathcal{M}$ *wraps a memory in a "pending operation" context.*
- *The unit* $\eta\colon \mathrm{id}_{\mathcal{M}} \Rightarrow T$ *embeds an unchanged memory as a trivial pending operation.*
- *The multiplication* $\mu\colon T^2 \Rightarrow T$ *flattens nested pending operations (e.g., evolving the result of an evolution) into a single pending operation.*

Proof. We verify the monad laws:
Left unit: $\mu \circ T\eta = \mathrm{id}_T$. Starting with a pending operation $T(m)$, wrapping it in a trivial outer context via $T\eta$ and then flattening via μ recovers $T(m)$, because η creates a no-op evolution and μ collapses the no-op.
Right unit: $\mu \circ \eta T = \mathrm{id}_T$. Starting with $T(m)$, embedding via η at the outer level and flattening gives back $T(m)$, because the outer wrapper is trivial.
Associativity: $\mu \circ T\mu = \mu \circ \mu T$. Given $T^3(m)$ (a triply-nested pending operation), flattening the inner two then the result equals flattening the outer two then the result. This holds because operation composition is associative: `evolve(evolve(evolve(m, r1), r2), r3)` produces the same final memory regardless of which pair of evolutions is collapsed first (the change reasons accumulate as a list, and list concatenation is associative). Concretely:

- `evolve`(m, r) creates a new version m' with m as parent, reason r, and incremented version number. This is T applied to m with context r.
- `merge`(m_1, m_2) creates m' with two parents. This is the tensor product in the monoidal structure.
- `split`(m) creates (m_1, m_2) from m. This is the comonadic dual, and together with merge provides the bimonad structure.

□

Remark 5.5. The merge and split operations extend the monad to a bimonad when we consider the comonad structure of split (extract = project to either component, duplicate = split each component further). The full bimonadic structure ensures that agent memory supports both accumulation (merge) and decomposition (split) of knowledge, which is essential for multi-agent collaboration.

6 Storage Architecture

6.1 Multi-Backend Router as Coproduct

ContextFS employs multiple storage backends, each optimized for different query patterns. The `StorageRouter` coordinates across them.

Definition 6.1 (Storage Router)**.** The *storage router* is the coproduct (disjoint union) of backend objects in $\mathcal{S}t$:

$$\text{Router} = \coprod_i B_i = B_{\text{SQLite}} \sqcup B_{\text{Chroma}} \sqcup B_{\text{Falkor}} \sqcup B_{\text{Postgres}}$$

equipped with injection morphisms $\iota_i \colon B_i \to \text{Router}$ and a universal property: for any backend C with morphisms $f_i \colon B_i \to C$, there exists a unique $[f_1, \ldots, f_n] \colon \text{Router} \to C$.

In practice, the router dispatches operations based on a routing policy:

```
Input: Operation op, Schema S, Data d
Output: Result r
backends ← route(S, op);
foreach B ∈ backends do
    if op ∈ Cap(B) then
        r_B ← B.execute(op, S, d);
    end
end
r ← merge_results({r_B});
return r
```

Algorithm 1: StorageRouter dispatch algorithm.

The routing policy implements a fan-out/fan-in pattern:

- **Save**: writes to all relevant backends (SQLite for structured data, ChromaDB for embeddings, FalkorDB for graph edges).
- **Recall**: reads from the primary backend (SQLite or PostgreSQL) with the authoritative copy.
- **Search**: dispatches to the backend best suited for the query type (ChromaDB for semantic, FalkorDB for graph traversal, SQLite for exact match).

6.2 Semantic Search

Semantic search maps queries to the embedding space and performs approximate nearest neighbor retrieval.

Definition 6.2 (Embedding Functor)**.** The *embedding functor* $E \colon \mathcal{S} \to \mathbf{Vect}_{\mathbb{R}}$ maps each schema S to a vector space $\mathbb{R}^d$ (where d is the embedding dimension) by:

$$E(S) = \text{embed}(\text{text}(S)) \in \mathbb{R}^d$$

where $\text{text} \colon \mathcal{S} \to \text{String}$ extracts a textual representation and $\text{embed} \colon \text{String} \to \mathbb{R}^d$ is the embedding model.

Semantic search then becomes inner product (or cosine similarity) computation in $\mathbb{R}^d$:

$$\texttt{search}(q, k) = \text{top}_k \left\{ m \in \mathsf{Mem}[-] \;\middle|\; \frac{\langle E(q), E(m) \rangle}{\|E(q)\| \cdot \|E(m)\|} > \theta \right\}$$

6.3 Graph Traversal

Graph queries in FalkorDB are formalized as path compositions in the edge relation category Edge.

Definition 6.3 (Path Query)**.** A *path query* π specifies a pattern of edges:

$$\pi = r_1 \circ r_2 \circ \cdots \circ r_n$$

where each $r_i \in \mathrm{mor}(\mathsf{Edge})$ is an edge relation. The result of executing π starting from memory m is the set of all memories reachable via paths matching the pattern.

Graph traversal composes with semantic search via the router:

$$\begin{tikzcd}
\text{Query} \arrow[r, "\text{embed}"] \arrow[d, "\text{parse}"'] & \mathbb{R}^d \arrow[r, "\text{ANN}"] & \text{Candidates} \arrow[d, "\text{filter}"] \\
\text{Graph Pattern} \arrow[rr, "\text{traverse}"'] & & \text{Results}
\end{tikzcd}$$

6.4 Capability Preservation

Theorem 6.4 (StorageRouter Preserves Capabilities)**.** *The capability functor* $\mathrm{Cap}\colon \mathcal{S}t \to \mathcal{P}(\mathrm{Ops})$ *satisfies:*

$$\mathrm{Cap}(\mathrm{Router}) = \bigcup_i \mathrm{Cap}(B_i)$$

That is, the router exposes the union of all backend capabilities.

Proof. By the coproduct construction, the router inherits each backend's capabilities through the injection morphisms ι_i. For any operation $\mathsf{op} \in \mathrm{Cap}(B_j)$, the router can dispatch to B_j via ι_j. Conversely, the router cannot perform an operation not supported by any backend, since it has no intrinsic computational capability—it only delegates. Thus $\mathrm{Cap}(\mathrm{Router}) = \bigcup_i \mathrm{Cap}(B_i)$. □

7 Reliability Guarantees

7.1 Vector Clocks for Causal Consistency

In multi-agent settings, memories may be updated concurrently by different agents. ContextFS uses vector clocks (**??**) to establish causal ordering.

Definition 7.1 (Vector Clock)**.** A *vector clock* for n agents is a function $\mathrm{vc}\colon \mathrm{Agents} \to \mathbb{N}$, where $\mathrm{vc}(a)$ represents the number of events agent a has processed. The partial order on vector clocks is:

$$\mathrm{vc}_1 \leq \mathrm{vc}_2 \iff \forall a.\ \mathrm{vc}_1(a) \leq \mathrm{vc}_2(a)$$

Two clocks are *concurrent* ($\mathrm{vc}_1 \| \mathrm{vc}_2$) when neither $\leq$ the other.

Each memory write increments the writing agent's component:

$$\mathrm{vc}'(a) = \begin{cases} \mathrm{vc}(a) + 1 & \text{if } a = \text{writer} \\ \max(\mathrm{vc}(a), \mathrm{vc}_{\text{received}}(a)) & \text{otherwise} \end{cases}$$

When two versions are concurrent, the system flags a conflict requiring resolution (via the merge operation or an explicit correction).

7.2 Content Hashing for Deduplication

ContextFS computes a content hash for every memory instance:

$$h(m) = \text{SHA256}(\text{canonical}(m.\texttt{data}))$$

where canonical produces a deterministic serialization (sorted keys, normalized whitespace). Two memories with the same content hash are identified as duplicates, regardless of when or by whom they were created.

Proposition 7.2 (Deduplication Soundness)**.** *Content hashing is sound for deduplication: if $h(m_1) = h(m_2)$, then $m_1.\texttt{data} = m_2.\texttt{data}$ with overwhelming probability ($1 - 2^{-256}$).*

7.3 Soft Deletes as Filtered Subcategory

ContextFS never physically deletes memories. Instead, it marks them with a tombstone:

Definition 7.3 (Tombstone Subcategory)**.** The *active memory subcategory* $\mathcal{M}_{\text{active}} \hookrightarrow \mathcal{M}$ is the full subcategory on memories whose `deleted_at` field is `nil`. All query operations implicitly filter to $\mathcal{M}_{\text{active}}$, while administrative operations have access to the full category $\mathcal{M}$.

This tombstone pattern preserves the complete lineage graph: even "deleted" memories retain their edges, allowing historical analysis and potential recovery.

7.4 ACID Properties

For the Elixir/OTP implementation, Mnesia transactions provide ACID guarantees:

1. **Atomicity**: Mnesia transactions are all-or-nothing; a failed write rolls back all changes.
2. **Consistency**: schema validation (via struct enforcement and typespecs) ensures that only valid memory instances are persisted.
3. **Isolation**: Mnesia's transaction manager serializes concurrent writes to the same memory.
4. **Durability**: Mnesia's disc_copies table type writes to both RAM and disk, surviving node restarts.

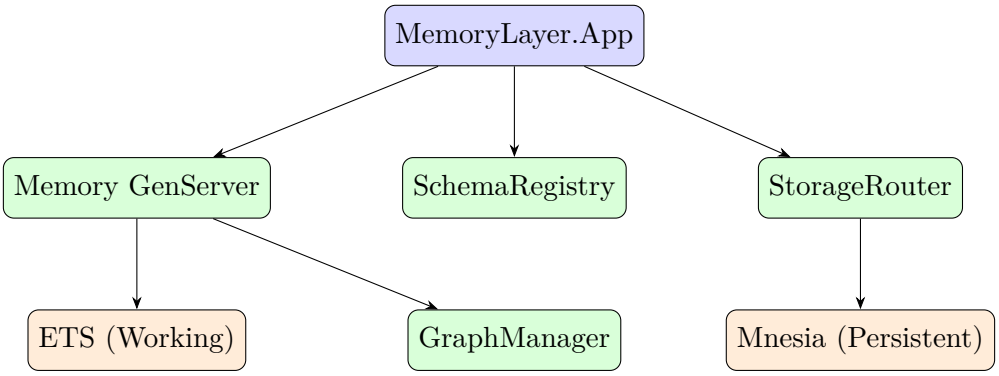
Proposition 7.4 (Transaction Safety)**.** *Every storage operation in the Elixir implementation is wrapped in a Mnesia transaction, ensuring that concurrent memory operations by multiple agents are serializable and that partial failures do not corrupt the memory graph.*

8 Elixir/OTP Implementation

We now present the Elixir/OTP reference implementation of the memory layer. The implementation leverages the BEAM VM's strengths: lightweight processes for per-memory GenServers, ETS for fast in-process working memory, Mnesia for distributed persistent storage, and OTP supervision trees for fault tolerance.

8.1 Architecture Overview

The implementation consists of seven modules organized in a supervision tree:



8.2 Schema Definitions

Schemas are defined as Elixir structs with enforce keys and typespecs, registered at application startup:

```
defmodule MemoryLayer.Schema do
  @type memory_type ::
    :fact | :decision | :procedural | :episodic |
    :todo | :issue | :api | :schema_def |
    :workflow | :task | :step | :agent_run |
    :tag | :annotation | :embedding | :index |
    :config | :session | :context | :log |
    :person | :project | :artifact | :event

  defmodule BaseSchema do
    @enforce_keys [:schema_name, :content]
    defstruct [
      :schema_name, :content, :metadata,
      id: nil, created_at: nil, updated_at: nil
    ]
    @type t :: %__MODULE__{
      schema_name: String.t(),
      content: map(),
      metadata: map() | nil,
      id: String.t() | nil,
      created_at: DateTime.t() | nil,
      updated_at: DateTime.t() | nil
    }
  end

  defmodule DecisionData do
    @enforce_keys [:decision, :rationale]
    defstruct [
      :decision, :rationale, :alternatives,
      :confidence, :context,
      schema_name: "decision"
    ]
```

```
  end
end
```

Listing 2: Schema definition and registry (abbreviated).

8.3 Typed Memory GenServer

The `Memory` module implements $\mathsf{Mem}[S]$ as a GenServer that holds typed state:

```
defmodule MemoryLayer.Memory do
  use GenServer
  alias MemoryLayer.{Schema, Version, Storage}

  @type t(schema) :: %{
    id: String.t(),
    schema_name: String.t(),
    data: schema,
    version: non_neg_integer(),
    content_hash: String.t(),
    created_at: DateTime.t(),
    updated_at: DateTime.t(),
    deleted_at: DateTime.t() | nil
  }

  # --- Public API (Mem[S] interface) ---

  @spec create(schema :: struct()) ::
    {:ok, pid()} | {:error, term()}
  def create(schema_data) do
    GenServer.start_link(__MODULE__, schema_data)
  end

  @spec data(pid()) :: {:ok, struct()} | {:error, term()}
  def data(pid), do: GenServer.call(pid, :get_data)

  @spec update(pid(), map()) :: :ok | {:error, term()}
  def update(pid, changes) do
    GenServer.call(pid, {:update, changes})
  end

  @spec evolve(pid(), map(), Version.change_reason()) ::
    {:ok, pid()} | {:error, term()}
  def evolve(pid, changes, reason) do
    GenServer.call(pid, {:evolve, changes, reason})
  end

  # --- GenServer callbacks ---

  @impl true
  def init(schema_data) do
    id = UUID.uuid4()
    now = DateTime.utc_now()
```

```
    hash = content_hash(schema_data)

    state = %{
      id: id,
      schema_name: schema_name(schema_data),
      data: schema_data,
      version: 1,
      content_hash: hash,
      created_at: now,
      updated_at: now,
      deleted_at: nil
    }

    # Persist to ETS (working memory)
    :ets.insert(:memory_working, {id, state})
    # Persist to storage backend
    Storage.save(state)

    {:ok, state}
  end

  @impl true
  def handle_call(:get_data, _from, state) do
    {:reply, {:ok, state.data}, state}
  end

  @impl true
  def handle_call({:update, changes}, _from, state) do
    updated_data = struct(state.data, changes)
    new_state = %{state |
      data: updated_data,
      version: state.version + 1,
      content_hash: content_hash(updated_data),
      updated_at: DateTime.utc_now()
    }
    :ets.insert(:memory_working, {state.id, new_state})
    Storage.update(new_state)
    {:reply, :ok, new_state}
  end

  @impl true
  def handle_call({:evolve, changes, reason}, _from, state) do
    # Create new memory, linked to parent
    evolved = struct(state.data, changes)
    {:ok, child_pid} = create(evolved)
    {:ok, child_state} = data(child_pid)
    Version.record(state.id, child_state.id, reason)
    {:reply, {:ok, child_pid}, state}
  end
end
```

Listing 3: Typed Memory GenServer (core operations).

8.4 Versioning

The `Version` module tracks causal history with change reason annotations:

```
defmodule MemoryLayer.Version do
  @type change_reason ::
    :observation | :inference | :correction | :decay

  @type version_entry :: %{
    memory_id: String.t(),
    parent_id: String.t() | nil,
    version: non_neg_integer(),
    reason: change_reason(),
    vector_clock: map(),
    timestamp: DateTime.t()
  }

  @spec record(parent_id :: String.t(),
               child_id :: String.t(),
               reason :: change_reason()) :: :ok
  def record(parent_id, child_id, reason) do
    entry = %{
      memory_id: child_id,
      parent_id: parent_id,
      version: next_version(parent_id),
      reason: reason,
      vector_clock: increment_clock(parent_id),
      timestamp: DateTime.utc_now()
    }
    :mnesia.transaction(fn ->
      :mnesia.write({:versions, child_id, entry})
    end)
    :ok
  end

  @spec history(memory_id :: String.t()) ::
    [version_entry()]
  def history(memory_id) do
    {:atomic, entries} = :mnesia.transaction(fn ->
      :mnesia.match_object({:versions, memory_id, :_})
    end)
    entries
    |> Enum.map(fn {:versions, _, entry} -> entry end)
    |> Enum.sort_by(& &1.timestamp)
  end
end
```

Listing 4: Version tracking with change reasons.

8.5 Graph Relationships

The graph module implements the edge relation category:

```elixir
defmodule MemoryLayer.Graph do
  @type edge_relation ::
    :evolved_into | :references | :supersedes |
    :contradicts | :supports | :derived_from |
    :part_of | :triggers | :blocked_by

  @spec link(from :: String.t(), to :: String.t(),
             relation :: edge_relation(),
             metadata :: map()) :: :ok
  def link(from_id, to_id, relation, metadata \\ %{}) do
    edge = %{
      from: from_id,
      to: to_id,
      relation: relation,
      metadata: metadata,
      created_at: DateTime.utc_now()
    }
    :mnesia.transaction(fn ->
      :mnesia.write({:edges, {from_id, to_id}, edge})
    end)
    :ok
  end

  @spec traverse(start :: String.t(),
                 pattern :: [edge_relation()],
                 opts :: keyword()) :: [String.t()]
  def traverse(start_id, pattern, opts \\ []) do
    max_depth = Keyword.get(opts, :max_depth, 10)
    do_traverse([start_id], pattern, max_depth, [])
  end
end
```

Listing 5: Graph relationship management.

8.6 Storage Backend

The storage router coordinates ETS (working memory) and Mnesia (persistent knowledge):

```elixir
defmodule MemoryLayer.Storage do
  @behaviour MemoryLayer.StorageBackend

  @callback save(memory :: map()) ::
    :ok | {:error, term()}
  @callback recall(id :: String.t()) ::
    {:ok, map()} | {:error, :not_found}
  @callback search(query :: String.t(), opts :: keyword()) ::
    {:ok, [map()]} | {:error, term()}
  @callback delete(id :: String.t()) ::
    :ok | {:error, term()}
  @callback update(memory :: map()) ::
```

```
    :ok | {:error, term()}

  # Fan-out save: ETS + Mnesia
  @impl true
  def save(memory) do
    :ets.insert(:memory_working, {memory.id, memory})
    :mnesia.transaction(fn ->
      :mnesia.write({:memories, memory.id, memory})
    end)
    :ok
  end

  # Recall from ETS first (fast path), fall back to Mnesia
  @impl true
  def recall(id) do
    case :ets.lookup(:memory_working, id) do
      [{^id, memory}] -> {:ok, memory}
      [] -> recall_from_mnesia(id)
    end
  end

  # Search across backends
  @impl true
  def search(query, opts \\ []) do
    backend = Keyword.get(opts, :backend, :all)
    case backend do
      :exact    -> search_mnesia(query, opts)
      :semantic -> search_embeddings(query, opts)
      :graph    -> search_graph(query, opts)
      :all      -> merge_results([
        search_mnesia(query, opts),
        search_embeddings(query, opts)
      ])
    end
  end
end
```

Listing 6: Multi-backend StorageRouter.

8.7 Working Memory vs. Persistent Knowledge

The dual-layer architecture mirrors human cognitive architecture:

- **ETS (Working Memory)**: in-process, microsecond access, lost on process crash. Holds the "hot" set of memories currently relevant to the agent's task. Analogous to working memory in cognitive psychology (**?**).
- **Mnesia (Persistent Knowledge)**: distributed, disk-backed, survives restarts. Holds the complete memory store. Analogous to long-term memory.

The promotion/demotion protocol between layers is:

1. On `create`: written to both ETS and Mnesia.
2. On `recall`: if in ETS, return immediately; if only in Mnesia, load into ETS (promotion).
3. On `evict`: remove from ETS but retain in Mnesia (demotion).
4. ETS is bounded; an LRU eviction policy demotes the least-recently-accessed memories.

8.8 MCP Server Interface

ContextFS exposes memory operations via the Model Context Protocol (MCP) (**?**), enabling any MCP-compatible client to access the memory layer:

Table 2: MCP tools exposed by ContextFS.

Tool	Description
`contextfs_save`	Create and persist a new typed memory
`contextfs_recall`	Retrieve a memory by ID with full type reconstruction
`contextfs_search`	Multi-backend search (exact, semantic, graph)
`contextfs_evolve`	Create a new version linked to parent with change reason
`contextfs_link`	Establish a typed edge between two memories
`contextfs_delete`	Soft-delete (tombstone) a memory
`contextfs_update`	Update fields of an existing memory
`contextfs_list`	List memories by type, tag, or time range
`contextfs_index`	Trigger re-indexing of embedding vectors
`contextfs_sync`	Synchronize local and cloud backends

The MCP server translates tool calls into GenServer messages, ensuring that all access goes through the typed memory interface and benefits from the type safety guarantees of Theorem **??**.

9 Case Studies

9.1 Agent Learning from Past Executions

Consider a testing agent that executes web application tests across multiple sessions. Without persistent memory, each session starts tabula rasa. With the memory layer:

1. After each test run, the agent creates `EPISODIC` memories recording what was tested, what failed, and what succeeded.
2. Failed test patterns are stored as `FACT` memories with schema fields for failure type, frequency, and suspected cause.
3. When the agent encounters a similar failure in a subsequent session, semantic search retrieves relevant past episodes, and the agent applies learned strategies.

4. Over time, `PROCEDURAL` memories accumulate: "When test X fails with error Y, the root cause is usually Z."

The version lineage tracks how the agent's understanding evolves: an initial observation ("test failed") evolves via inference ("this is a race condition") and may later be corrected ("actually, it was a configuration issue").

```
# Session N: Agent encounters a test failure
{:ok, episode} = Memory.create(%EpisodicData{
  event: "login_test_timeout",
  context: %{url: "https://app.example.com/login"},
  outcome: :failure,
  duration_ms: 30_000
})

# Semantic search for similar past failures
{:ok, similar} = Storage.search(
  "login test timeout failure",
  backend: :semantic, limit: 5
)

# Found! Previous session recorded a procedural fix
# Apply the fix and record the evolution
{:ok, _} = Memory.evolve(episode, %{
  outcome: :success,
  resolution: "Increased timeout to 60s per PROC-442"
}, :correction)
```

Listing 7: Agent learning from past test failures.

9.2 Cross-Session Continuity

A software engineering agent works on a multi-day project. Between sessions, the user may make changes that the agent needs to integrate:

1. At session end, the agent stores its current context as a `SESSION` memory, including active tasks, pending decisions, and environmental assumptions.
2. At session start, the agent recalls its last session memory and all `TODO` items marked as in-progress.
3. The agent detects that some assumptions (stored as `FACT` memories) may have been invalidated by external changes. It creates new versions with `Correction` reasons.
4. The graph lineage shows the full narrative: original plan → partial execution → external change → plan revision.

```
# Session start: recall last session state
{:ok, sessions} = Storage.search(
  "session_state",
  backend: :exact,
```

```
  filter: %{schema_name: "session"},
  sort: :updated_at_desc,
  limit: 1
)

last_session = hd(sessions)

# Recall all in-progress TODOs
{:ok, todos} = Storage.search(
  "status:in_progress",
  backend: :exact,
  filter: %{schema_name: "todo"}
)

# Check for invalidated assumptions
for todo <- todos do
  deps = Graph.traverse(todo.id, [:references], max_depth: 2)
  for dep_id <- deps do
    {:ok, dep} = Storage.recall(dep_id)
    if stale?(dep), do: flag_for_review(dep)
  end
end
```

Listing 8: Cross-session context restoration.

9.3 Multi-Agent Shared Knowledge Base

Multiple agents collaborate on a shared codebase, each responsible for different modules:

1. Agent A (frontend) discovers a bug in an API contract and creates an `ISSUE` memory.
2. Agent B (backend) subscribes to `ISSUE` memories via Mnesia replication and receives the notification.
3. Agent B investigates, creates a `DECISION` memory ("change the API response format"), linked to the original issue via `DERIVED_FROM`.
4. Agent A detects the decision via graph traversal from its original issue, creates `TASK` memories to update the frontend, and links them via `TRIGGERS`.

Vector clocks ensure that concurrent edits to shared memories are detected:

```
# Agent A writes with its clock component
vc_a = %{agent_a: 5, agent_b: 3}

# Agent B writes concurrently
vc_b = %{agent_a: 4, agent_b: 4}

# Neither dominates: vc_a[a]=5 > vc_b[a]=4, but
#                    vc_a[b]=3 < vc_b[b]=4
# Conflict detected! Requires merge resolution.
```

```
case Version.compare_clocks(vc_a, vc_b) do
  :concurrent ->
    # Flag for merge resolution
    Graph.link(mem_a.id, mem_b.id, :contradicts,
      %{resolution: :pending})
  :before -> :ok   # vc_a happened-before vc_b
  :after -> :ok    # vc_b happened-before vc_a
end
```

Listing 9: Multi-agent conflict detection.

10 Discussion

10.1 Comparison to Unix VFS

The Unix Virtual Filesystem Switch (VFS) and our StorageRouter share the goal of providing a uniform interface over heterogeneous storage backends. However, the VFS operates at the byte-stream level: `read(fd, buf, count)` makes no type promises about the bytes returned. Our StorageRouter operates at the typed-schema level: `recall(id)` returns a $\mathsf{Mem}[S]$ with compile-time and runtime type guarantees.

The VFS uses inodes as the unit of storage; we use typed memory instances. The VFS organizes files in a tree hierarchy; we organize memories in a labeled graph. The VFS supports a fixed set of operations (`open`, `read`, `write`, `close`, `stat`); our `StorageBackend` protocol supports extensible operation sets including semantic search.

10.2 Comparison to Plan 9

Plan 9's insight was that *everything should be a file*—even network connections, process state, and device control. Our analogous principle is that *everything an agent knows should be a typed memory*—including its own execution traces (`AGENT_RUN`), its task queue (`TODO`), and its configuration (`CONFIG`).

Plan 9 used synthetic file servers (e.g., `/proc`) to present dynamic state as files. We use MCP tools (e.g., `contextfs_search`) to present dynamic memory state as structured, queryable knowledge. The key advance is that our "files" carry schemas, versions, and graph relationships.

10.3 Comparison to Semantic Filesystems

Semantic filesystems (**??**) pioneered the idea of retrieving files by metadata attributes rather than path names. Our semantic search extends this to *meaning-based retrieval* via embeddings, which captures similarity that metadata tags cannot express.

Moreover, semantic filesystems typically operate on a flat attribute space; our memory layer adds graph structure and causal versioning.

10.4 Relation to Knowledge Graphs

Knowledge graphs (e.g., Wikidata, Neo4j-backed enterprise graphs) store entities and relationships but typically lack:

- **Schema-indexed typing**: KG nodes are typically loosely typed with optional properties. Our $\mathsf{Mem}[S]$ enforces strict schemas with compile-time checking.
- **Causal versioning**: KGs store current state; versioning is an external concern. Our $\mathsf{VMem}[S]$ integrates versioning as a first-class functor.
- **Embedding integration**: KGs and vector stores are typically separate systems. Our StorageRouter unifies them.

10.5 Limitations and Future Work

Several limitations merit future investigation:

1. **Garbage collection**: the tombstone pattern means storage grows monotonically. A formal garbage collection policy (analogous to generational GC in runtime systems) would bound storage while preserving essential lineage.
2. **Schema evolution**: when a schema's definition changes (fields added/removed), existing memories must be migrated. Our framework handles this via schema morphisms, but the migration strategy needs automation.
3. **Scalability**: Mnesia's distribution model has known limitations beyond a few hundred nodes. For planetary-scale agent deployments, a CockroachDB or FoundationDB backend may be necessary.
4. **Embedding drift**: as embedding models are updated, previously computed vectors may become inconsistent with new ones. Re-indexing strategies need formal treatment.
5. **Privacy and forgetting**: the "right to be forgotten" conflicts with the tombstone pattern. A cryptographic erasure scheme (encrypt memory content with a key that can be destroyed) would reconcile these requirements.

11 Related Work

The memory layer draws on several research traditions.

Agent architectures with memory. Soar (**?**) distinguished working memory, long-term declarative, procedural, and episodic memory—a taxonomy our 24 types refine and extend. ACT-R (**?**) formalized memory retrieval as activation-based competition, which inspired our embedding-based semantic search. Generative agents (**?**) demonstrated that LLM agents benefit from structured memory stores, though their implementation lacked type safety and formal versioning.

Type-theoretic databases. Categorical databases (CQL) (**??**) formalize database schemas as categories and instances as functors, closely paralleling our construction. Our contribution is adapting this framework to *agent memory* with versioning, lineage, and multi-modal storage.

Version control. Git (**?**) uses content-addressable storage with a DAG of commits. Our version lineage is analogous but operates on typed memory instances rather than file trees, and our change reasons provide semantic annotations that Git commit messages cannot enforce structurally.

Distributed consistency. Dynamo (**?**) popularized vector clocks for eventually consistent storage. Our adaptation applies vector clocks to agent memory with the additional constraint that conflicts must be resolved through typed merge operations (not arbitrary last-writer-wins).

Memory in LLM systems. MemGPT (**?**) introduced OS-inspired memory tiers for LLMs. Langchain's memory modules (**?**) provide buffer, summary, and entity memory. Our framework subsumes these with formal type guarantees and a categorical foundation.

12 Conclusion

We have presented a categorical framework for agent memory that formalizes the memory layer of an AI Operating System. The key contributions are:

1. A **formal memory model** based on the functor $M : \mathcal{S} \to \mathcal{S}t$, with the Yoneda-lemma connection establishing $\mathsf{Mem}[S]$ as a representable functor (Theorem **??**).
2. A **type system** with 24 schema-indexed memory types and operations expressed as natural transformations, with a proven round-trip type-preservation theorem (Theorem **??**).
3. A **versioning framework** based on the temporal functor $V : \mathsf{Timeline} \to \mathcal{S}t$, with graph lineage as 2-categorical structure and memory operations forming a monad (Proposition **??**).
4. A **multi-backend storage architecture** formalized as a coproduct with proven capability preservation (Theorem **??**).
5. A **production implementation** in Elixir/OTP using ETS and Mnesia, with Mnesia transactions providing ACID guarantees, and an MCP server enabling universal client access.
6. **Case studies** demonstrating agent learning, cross-session continuity, and multi-agent collaboration.

The memory layer is the third component of the AI Operating System, building on the kernel (Part I) and scheduler (Part II). Together, these three components provide process management, resource allocation, and persistent knowledge—the foundations upon which higher-level agent capabilities (planning, tool use, collaboration) can be constructed. Part IV will address the tool interface layer, formalizing how agents interact with external systems through typed, composable tool protocols.

The categorical perspective is not mere formalism: it provides the compositional guarantees that make the memory layer *trustworthy*. When an agent stores a decision and later recalls it, the type-preservation theorem ensures the recalled data conforms to the decision schema. When two agents concurrently update shared knowledge, vector clocks detect conflicts and the monad structure of merge/split ensures sound resolution. When the storage backend is extended with a new engine, the coproduct construction guarantees that existing capabilities are preserved.

Agent cognition requires memory that is typed, versioned, searchable, and graph-structured. The filesystem analogy is apt but insufficient: what agents need is

not a filesystem but a *knowledge system*. The categorical memory layer provides exactly this.

References

J. R. Anderson, D. Bothell, M. D. Byrne, S. Douglass, C. Lebiere, and Y. Qin. An integrated theory of the mind. *Psychological Review*, 111(4):1036–1060, 2004.

Anthropic. Model Context Protocol specification. Technical report, Anthropic, 2024. https://modelcontextprotocol.io.

F. Baader, D. Calvanese, D. L. McGuinness, D. Nardi, and P. F. Patel-Schneider. *The Description Logic Handbook: Theory, Implementation, and Applications*. Cambridge University Press, 2003.

A. Baddeley. Working memory. *Science*, 255(5044):556–559, 1992.

J. Bénabou. Introduction to bicategories. In *Reports of the Midwest Category Seminar*, volume 47 of *Lecture Notes in Mathematics*, pages 1–77. Springer, 1967.

S. Chacon and B. Straub. *Pro Git*. Apress, 2nd edition, 2014.

G. DeCandia, D. Hastorun, M. Jampani, G. Kakulapati, A. Lakshman, A. Pilchin, S. Sivasubramanian, P. Vosshall, and W. Vogels. Dynamo: Amazon's highly available key-value store. In *Proc. 21st ACM SOSP*, pages 205–220, 2007.

P. J. Denning. Virtual memory. *ACM Computing Surveys*, 2(3):153–189, 1970.

J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proc. NAACL-HLT*, pages 4171–4186, 2019.

C. J. Fidge. Timestamps in message-passing systems that preserve the partial ordering. In *Proc. 11th Australian Computer Science Conference*, pages 56–66, 1988.

D. K. Gifford, P. Jouvelot, M. A. Sheldon, and J. W. O'Toole Jr. Semantic file systems. In *Proc. 13th ACM SOSP*, pages 16–25, 1991.

B. Gopal and U. Manber. Integrating content-based access mechanisms with hierarchical file systems. In *Proc. 3rd USENIX OSDI*, pages 265–278, 1999.

A. Hogan, E. Blomqvist, M. Cochez, C. d'Amato, G. de Melo, C. Gutierrez, S. Kirrane, J. E. L. Gayo, R. Navigli, S. Neumaier, A.-C. N. Ngomo, A. Polleres, S. M. Rashid, A. Rula, L. Schmelzeisen, J. Sequeda, S. Staab, and A. Zimmermann. Knowledge graphs. *ACM Computing Surveys*, 54(4):1–37, 2021.

J. Johnson, M. Douze, and H. Jégou. Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*, 7(3):535–547, 2019.

M. J. Kahana. *Foundations of Human Memory*. Oxford University Press, 2012.

J. E. Laird. *The Soar Cognitive Architecture*. MIT Press, 2012.

L. Lamport. Time, clocks, and the ordering of events in a distributed system. *Communications of the ACM*, 21(7):558–565, 1978.

LangChain. LangChain documentation: Memory. https://docs.langchain.com/docs/components/memory, 2023.

P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Proc. NeurIPS*, pages 9459–9474, 2020.

R. Love. *Linux Kernel Development*. Addison-Wesley, 3rd edition, 2010.

S. Mac Lane. *Categories for the Working Mathematician*. Springer, 2nd edition, 1978.

M. Minsky. A framework for representing knowledge. MIT AI Laboratory Memo 306, 1974.

C. Packer, V. Fang, S. G. Patil, K. Lin, S. Wooders, and J. E. Gonzalez. MemGPT: Towards LLMs as operating systems. *arXiv preprint arXiv:2310.08560*, 2023.

J. S. Park, J. C. O'Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein. Generative agents: Interactive simulacra of human behavior. In *Proc. 36th ACM UIST*, pages 1–22, 2023.

R. Pike, D. Presotto, S. Dorward, B. Flandrena, K. Thompson, H. Trickey, and P. Winterbottom. Plan 9 from Bell Labs. *Computing Systems*, 8(3):221–254, 1995.

M. R. Quillian. Semantic memory. In M. Minsky, editor, *Semantic Information Processing*, pages 227–270. MIT Press, 1968.

N. Reimers and I. Gurevych. Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In *Proc. EMNLP-IJCNLP*, pages 3982–3992, 2019.

D. M. Ritchie and K. Thompson. The UNIX time-sharing system. *Communications of the ACM*, 17(7):365–375, 1974.

P. Schultz, D. I. Spivak, and C. Vasilakopoulou. Algebraic databases. *Theory and Applications of Categories*, 32(16):547–619, 2017.

D. I. Spivak. *Category Theory for the Sciences*. MIT Press, 2014.

J. Armstrong. *Programming Erlang: Software for a Concurrent World*. Pragmatic Bookshelf, 2nd edition, 2013.

R. Virding, C. Wikström, and M. Williams. *Concurrent Programming in Erlang*. Prentice Hall, 2nd edition, 1996.

D. Thomas and A. Hunt. *Programming Elixir $\geq$ 1.6*. Pragmatic Bookshelf, 2018.

H. Mattsson, H. Nilsson, and C. Wikström. Mnesia — A distributed robust DBMS for telecommunications applications. In *Proc. 1st International Workshop on Practical Aspects of Declarative Languages (PADL)*, pages 152–163, 1999.

M. Shapiro, N. Preguiça, C. Baquero, and M. Zawirski. Conflict-free replicated data types. In *Proc. 13th International Symposium on Stabilization, Safety, and Security of Distributed Systems (SSS)*, pages 386–400, 2011.

M. Kleppmann. *Designing Data-Intensive Applications.* O'Reilly Media, 2017.

T. Leinster. *Basic Category Theory.* Cambridge University Press, 2014.

E. Riehl. *Category Theory in Context.* Dover Publications, 2017.

S. Awodey. *Category Theory.* Oxford University Press, 2nd edition, 2010.

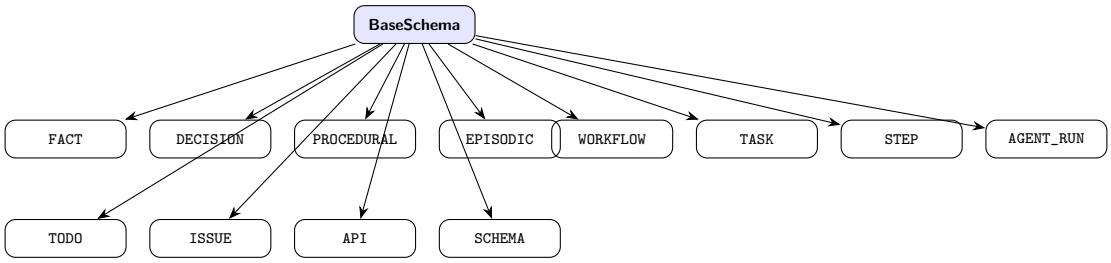
M. Barr and C. Wells. *Category Theory for Computing Science.* Prentice Hall, 1990.

E. Moggi. Notions of computation and monads. *Information and Computation*, 93(1):55–92, 1991.

P. Wadler. Monads for functional programming. In *Advanced Functional Programming*, volume 925 of *LNCS*, pages 24–52. Springer, 1992.

A Complete Memory Type Hierarchy

The following diagram shows the complete inheritance structure of the 24 memory types, organized as a directed acyclic graph with `BaseSchema` as the root:



B Commutative Diagrams for Storage Operations

The full round-trip diagram for save/recall across the multi-backend router:

$$\begin{tikzcd}
\mathsf{Mem}[S] \arrow[r, "\texttt{save}"] \arrow[d, "\texttt{embed}"'] \arrow[dr, phantom, "\circlearrowleft"] & B_{\text{SQLite}} \arrow[d, "\texttt{replicate}"] \\
B_{\text{Chroma}} \arrow[r, "\texttt{sync}"'] & B_{\text{Postgres}}
\end{tikzcd}$$

The versioning chain with change reasons:

$$\begin{tikzcd}
m_0 \arrow[r, "\text{Observation}"] & m_1 \arrow[r, "\text{Inference}"] & m_2 \arrow[r, "\text{Correction}"] & m_3 \\
 & & m_2' \arrow[ur, dashed, "\text{Merge}"'] &
\end{tikzcd}$$

C MCP Protocol Messages

Example MCP tool call for `contextfs_save`:

```
{
  "jsonrpc": "2.0",
  "method": "tools/call",
  "params": {
    "name": "contextfs_save",
    "arguments": {
      "content": "Login test requires 60s timeout",
      "memory_type": "fact",
      "metadata": {
        "confidence": 0.95,
        "source": "test_run_2026_03_10"
      },
      "tags": ["testing", "timeout", "login"]
    }
  },
  "id": 42
}
```

Listing 10: MCP tool call example (JSON-RPC).

D Proof: Monad Laws for Memory Operations (Detailed)

We provide the full verification of the monad laws for the memory operations endofunctor T.

Setup. Let $\mathcal{M}$ be the category of typed memory instances. Define $T\colon \mathcal{M} \to \mathcal{M}$ by:

$$T(m) = (m, \bot, \texttt{pending})$$

where $\bot$ denotes "no changes yet" and `pending` marks the operation as unevaluated. The unit $\eta_m = (m, \emptyset, \texttt{noop})$ wraps m in a trivial operation. The multiplication μ collapses $(T(m), \delta_1, r_1)$ with outer $(-, \delta_2, r_2)$ into $(m, \delta_1 \cup \delta_2, r_1 \sqcup r_2)$ where $\sqcup$ is the join in the reason lattice.

Left unit: $\mu_m \circ T(\eta_m) = \mu_m((m, \emptyset, \texttt{noop}), \bot, \texttt{pending}) = (m, \emptyset, \texttt{noop}) = T(m)$ after simplification (pending with no changes reduces to the inner state).

Right unit: $\mu_m \circ \eta_{T(m)} = \mu_m(\eta((m, \delta, r))) = \mu_m(((m, \delta, r), \emptyset, \texttt{noop})) = (m, \delta, r) = T(m)$.

Associativity: For $T^3(m)$ with changes $\delta_1, \delta_2, \delta_3$ and reasons r_1, r_2, r_3:

$$\mu \circ T\mu : (m, \delta_1 \cup \delta_2 \cup \delta_3, r_1 \sqcup r_2 \sqcup r_3)$$
$$\mu \circ \mu T : (m, \delta_1 \cup \delta_2 \cup \delta_3, (r_1 \sqcup r_2) \sqcup r_3)$$

These are equal by associativity of $\cup$ and $\sqcup$. $\square$